# DEPLOYMENT OF ONLINE TICKETING & EVENT MANAGEMENT INFRASTRUCTURE

*A DevOps CI/CD Implementation for a Scalable Movie-Ticketing Platform (BookMyShow)*

**Domain**

E-Commerce and Entertainment | Cloud Computing & DevOps

Prepared by: Parikshit A

Cloud Provider: Amazon Web Services (AWS)

Region: ap-south-1 (Asia Pacific — Mumbai)

Date: August 2026

# Table of Contents

1. Project Overview ..... 3
2. Architecture & Pipeline Workflow ..... 4
3. Networking & Security Foundation (VPC) ..... 4
4. Compute Tier — Jenkins EC2 Server ..... 6
5. Source Control & Containerization ..... 8
6. CI/CD Pipeline — Jenkins ..... 11
7. Container Registry — Amazon ECR ..... 12
8. Container Orchestration — Amazon EKS ..... 14
9. Configuration Management — Ansible Deployment ..... 16
10. Load Balancing & End-to-End Verification ..... 17
11. Identity & Access Management (IAM) ..... 18
12. Source Repository — GitHub ..... 19
13. Monitoring & Observability (Prometheus & Grafana) ..... 20
14. Summary & Outcomes ..... 22

# 1. Project Overview

BookMyShow is a popular online movie-ticketing platform that lets users browse movies, book tickets, and manage bookings. This project re-platforms a monolithic BookMyShow-style application onto a microservices deployment on AWS, replacing manual releases with a fully automated CI/CD pipeline. Unlike a purely manual AWS setup, this deployment leans on containerization and orchestration — Docker, Jenkins, Amazon ECR, Amazon EKS (Kubernetes), and Ansible — to make releases repeatable, scalable, and observable.

## 1.1 Source Repository

GitHub: github.com/ParikshitArer/Book-My-Show (forked from staragile2016/Book-My-Show)

## 1.2 Business Goals

- Deliver frequent and reliable updates to production.
- Accelerate software delivery speed while maintaining high quality.
- Reduce manual intervention in deployments and testing.
- Improve scalability and monitoring of the application.

## 1.3 Current Challenges Addressed

- Complex builds — managing dependencies and incremental builds manually.
- Manual testing — verifying multiple components by hand is slow and error-prone.
- Infrastructure setup — manual server provisioning delays releases.
- Monitoring — no real-time visibility meant issues surfaced late.

## 1.4 Tools & Technologies

| Tool | Role in the Pipeline |
|---|---|
| Git / GitHub | Version control; triggers the pipeline on push to main. |
| Jenkins | CI/CD orchestration — build, test, containerize, deploy. |
| Docker | Packages the app into a portable, multi-stage container image. |
| Amazon ECR | Private registry that stores versioned Docker images. |
| Amazon EKS (Kubernetes) | Runs and scales the containerized application. |
| Ansible | Drives the Kubernetes rollout from Jenkins (configuration-as-code). |
| AWS EC2 | Hosts the Jenkins controller / build agent. |
| Prometheus & Grafana | Collects and visualizes cluster and application metrics. |

*Table 1.1 — Tools and technologies used across the pipeline.*

# 2. Architecture & Pipeline Workflow

A GitHub webhook triggers Jenkins on every push to the main branch. Jenkins checks out the code, builds and tags a Docker image, pushes it to Amazon ECR, then hands off to Ansible to roll the new image out to the Amazon EKS cluster. Prometheus scrapes cluster and workload metrics, and Grafana renders them on live dashboards so the health of each release is visible immediately after deployment.

## 2.1 Pipeline Workflow

- GitHub webhook trigger — a push to main starts the Jenkins job automatically.
- Checkout & verify — Jenkins pulls the repository and validates expected files are present.
- Docker build — the multi-stage Dockerfile compiles the React/Node front end and packages it behind Nginx.
- ECR login & push — the tagged image is authenticated and pushed to the private ECR repository.
- Ansible deploy to EKS — Ansible updates the Kubernetes Deployment image and waits for rollout.
- Verify EKS deployment — pipeline confirms pods are Running and the Service has a healthy endpoint.
- Deployment validation — an HTTP check against the load balancer confirms the release is live.
- Monitoring — Prometheus and Grafana provide ongoing visibility into cluster and pod health.

## 2.2 Tier-to-Service Mapping

| Tier | AWS / DevOps Service | Role |
|---|---|---|
| Source Control | GitHub | Stores code; triggers CI/CD via webhook. |
| CI/CD | Jenkins (on EC2) | Automates build, test, containerize, deploy. |
| Containerization | Docker | Packages the app as a portable image. |
| Registry | Amazon ECR | Private, encrypted store for image versions. |
| Orchestration | Amazon EKS | Runs, scales, and self-heals the workload. |
| Configuration Mgmt | Ansible | Automates the Kubernetes rollout. |
| Networking | Amazon VPC, Security Groups | Isolates and secures cluster resources. |
| Identity | AWS IAM | Grants least-privilege access between services. |
| Load Balancing | Classic Load Balancer (K8s Service) | Distributes traffic across pods/AZs. |
| Monitoring | Prometheus & Grafana | Metrics collection and dashboards. |

*Table 2.1 — Mapping of pipeline stages to AWS/DevOps services.*

# 3. Networking & Security Foundation (VPC)

A dedicated VPC named Bookmyshow gives the EKS cluster network isolation. Two public subnets, spread across ap-south-1a and ap-south-1b, host the EKS worker nodes and load balancer, with a single route table and Internet Gateway providing outbound/inbound connectivity.

| | |
|---|---|
| **VPC Name** | Bookmyshow |
| **VPC ID** | vpc-0fe3a4ca7e5fbd5e2 |

| | |
|---|---|
| **Status** | Available |
| **Subnets** | 2 — Public-subnet-1 (ap-south-1a), Public-subnet-2 (ap-south-1b) |
| **Route Tables** | 2 — RT-1 (rtb-03032222fb5a998c4) |
| **Internet Gateway** | IGW — provides outbound/inbound internet access |

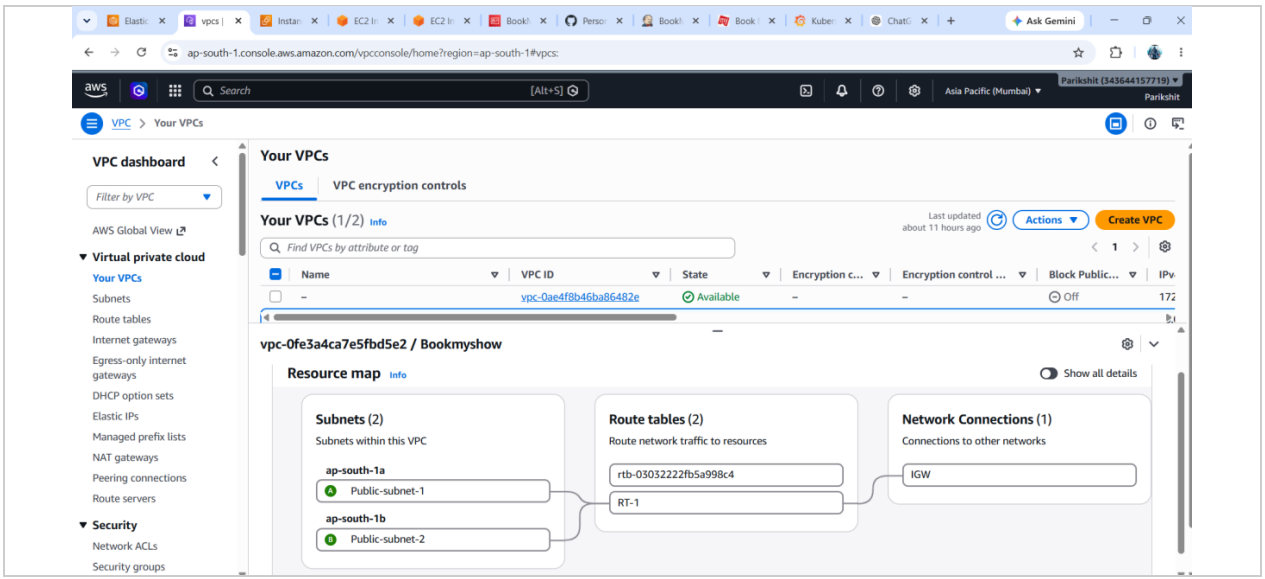



*Figure 1 — VPC resource map — subnets across two Availability Zones, route tables, and Internet Gateway.*

## 3.1 Security Groups

A dedicated security group, BookMyShow-EKS-SG, was created inside the VPC to control traffic to the EKS workloads. It is created with no inbound rules by default, following the principle of least privilege — rules are added only as each workload requires them.

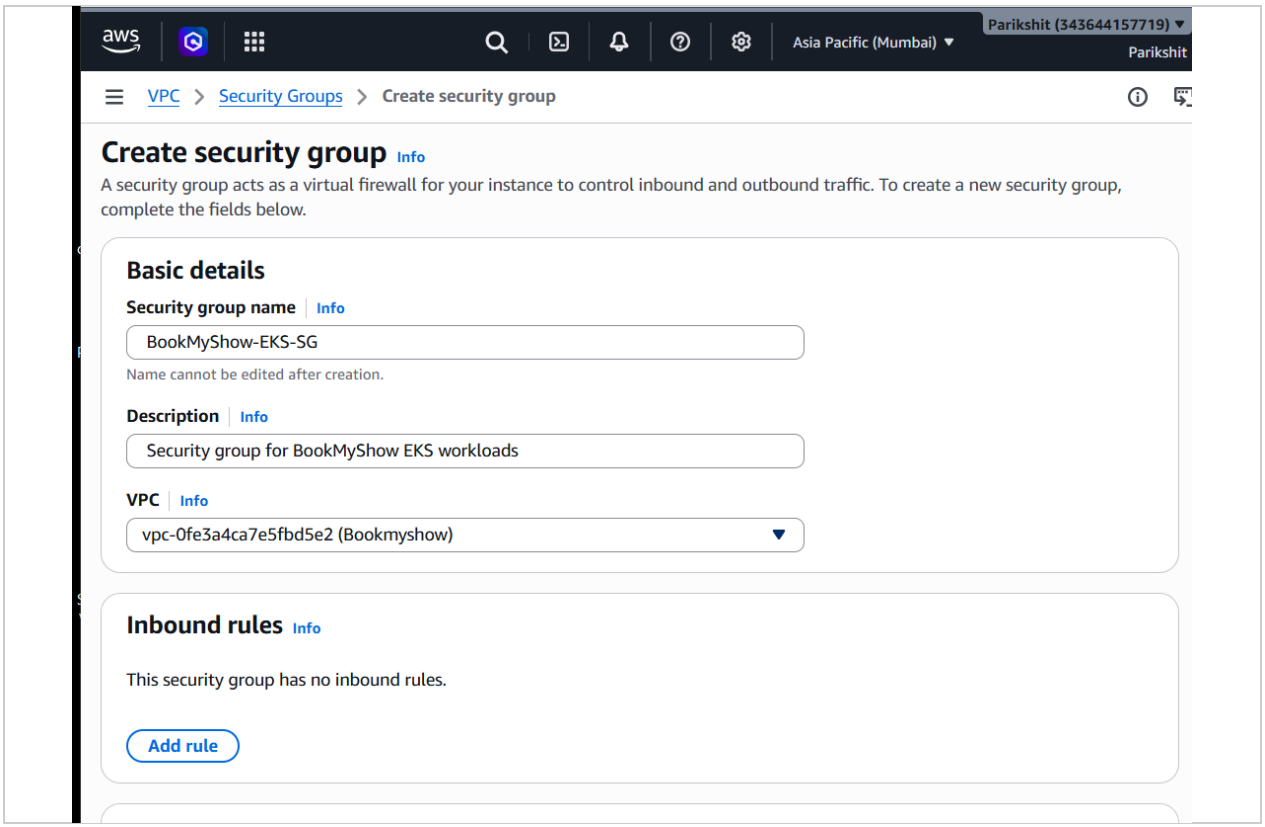



*Figure 2 — Creating the BookMyShow-EKS-SG security group inside the Bookmyshow VPC.*

# 4. Compute Tier — Jenkins EC2 Server

A single EC2 instance (Jenkins EC2, instance type c7i-flex.large) hosts the Jenkins controller that drives the entire pipeline. Two additional t3.small EC2 instances act as the EKS worker nodes.

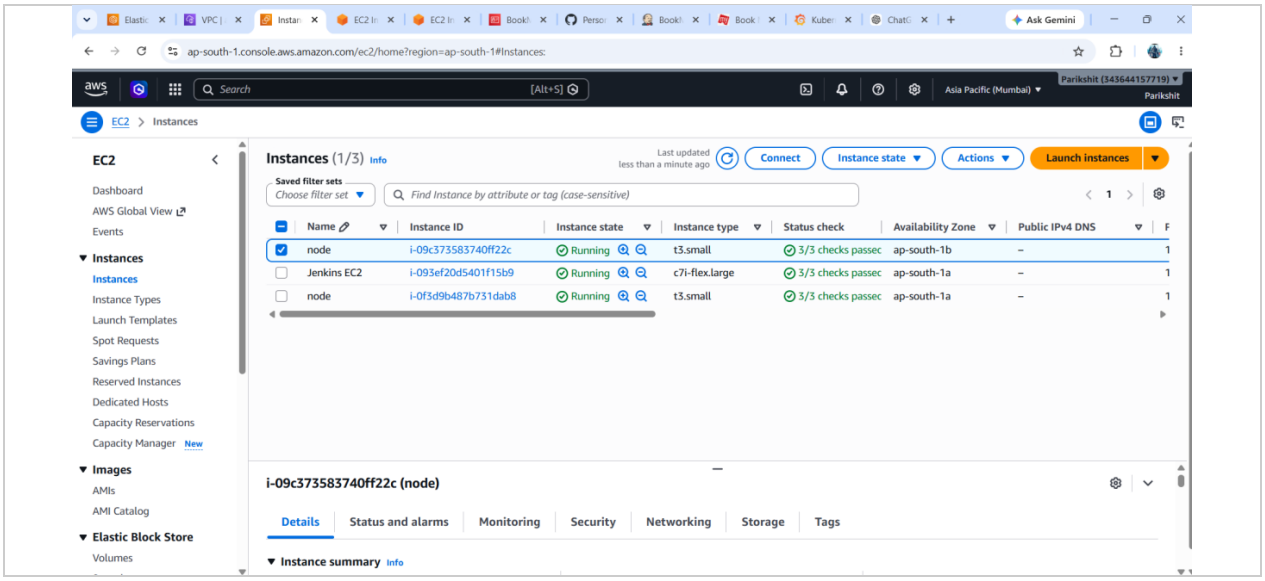



*Figure 3 — EC2 instances: the Jenkins controller and the EKS worker nodes, all in a Running state.*

## 4.1 Jenkins Service

Jenkins runs as a systemd service on the EC2 host and was verified active and running before configuring the pipeline.

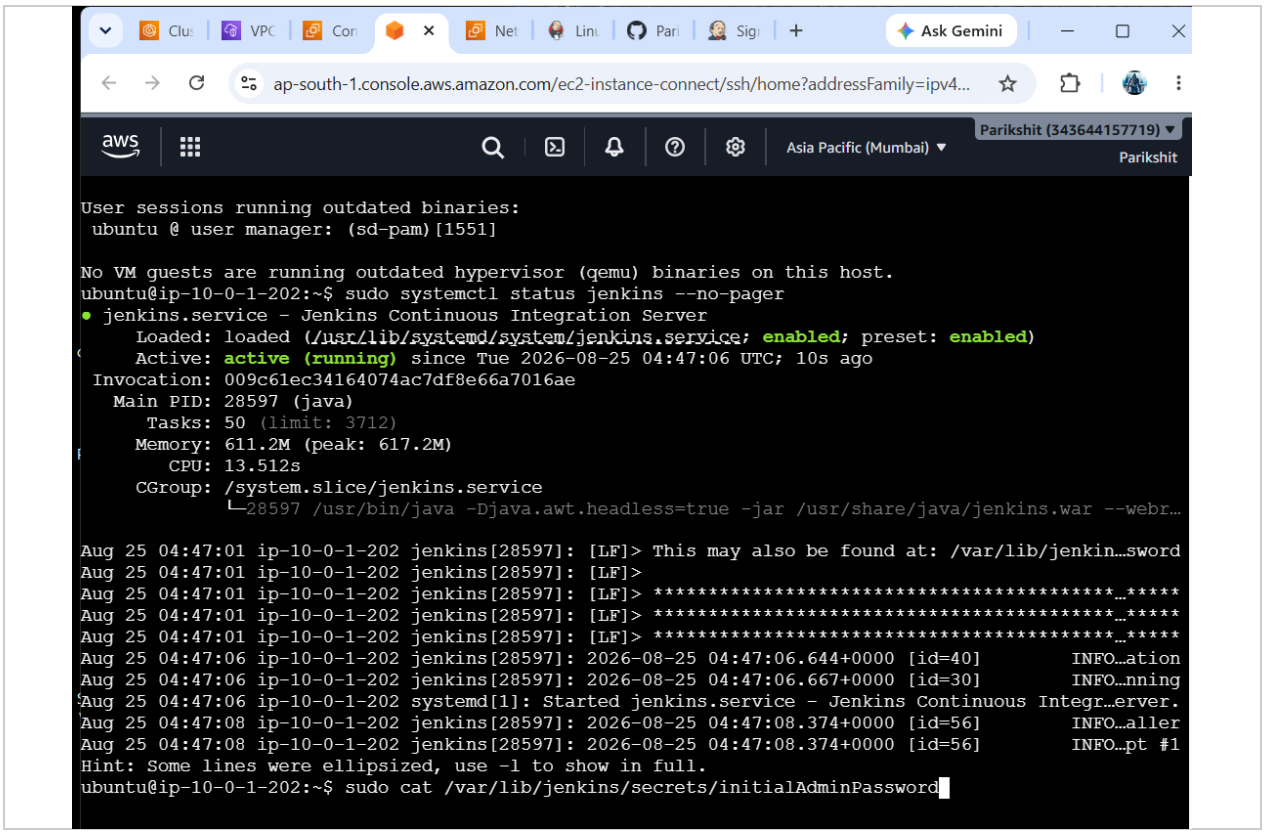



*Figure 4 — Jenkins service status confirmed active (running) via systemctl.*

## 4.2 Tooling & IAM Role Verification

Rather than storing long-lived AWS credentials on the box, the Jenkins EC2 instance assumes the BookMyShow-Jenkins-Roles IAM role. The build host was provisioned with the AWS CLI, kubectl, and Git, and aws sts get-caller-identity was used to confirm the instance is authenticating via the assumed role rather than static keys.

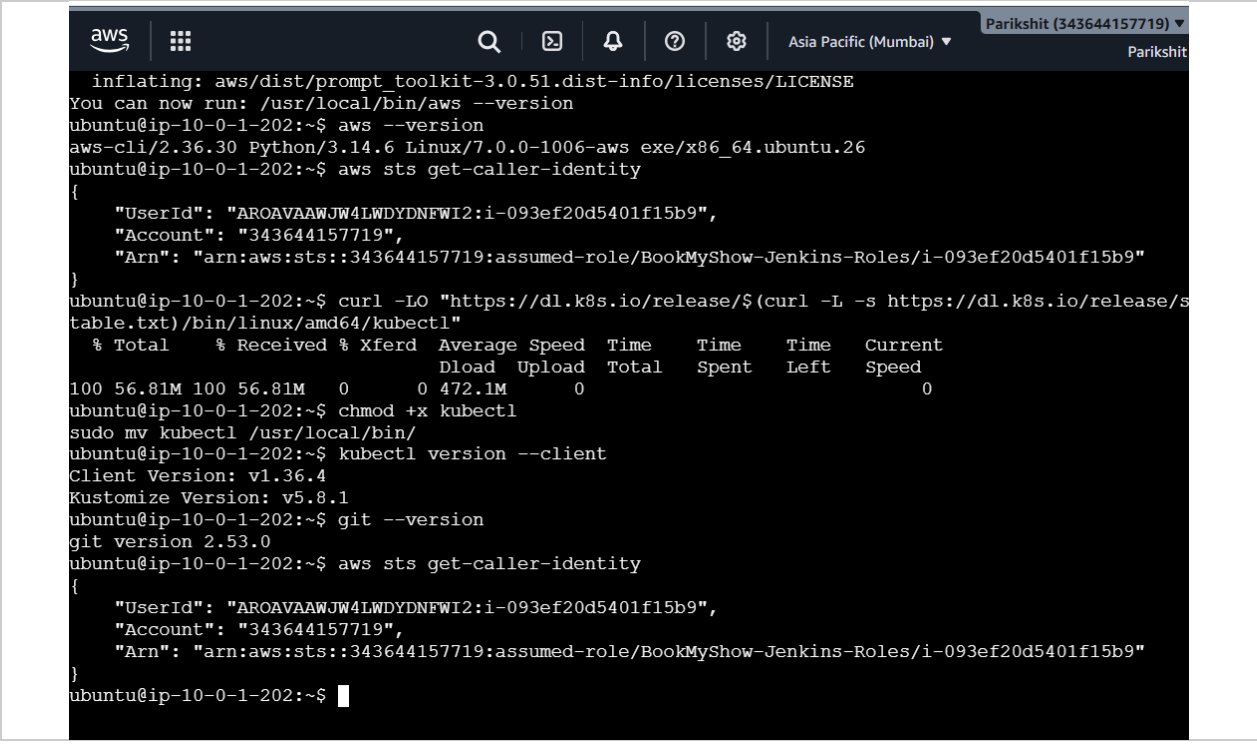



*Figure 5 — Installing kubectl and verifying the assumed IAM role via aws sts get-caller-identity.*

# 5. Source Control & Containerization

The application source is cloned directly onto the Jenkins host from the GitHub repository to confirm the pipeline's checkout step ahead of automation.

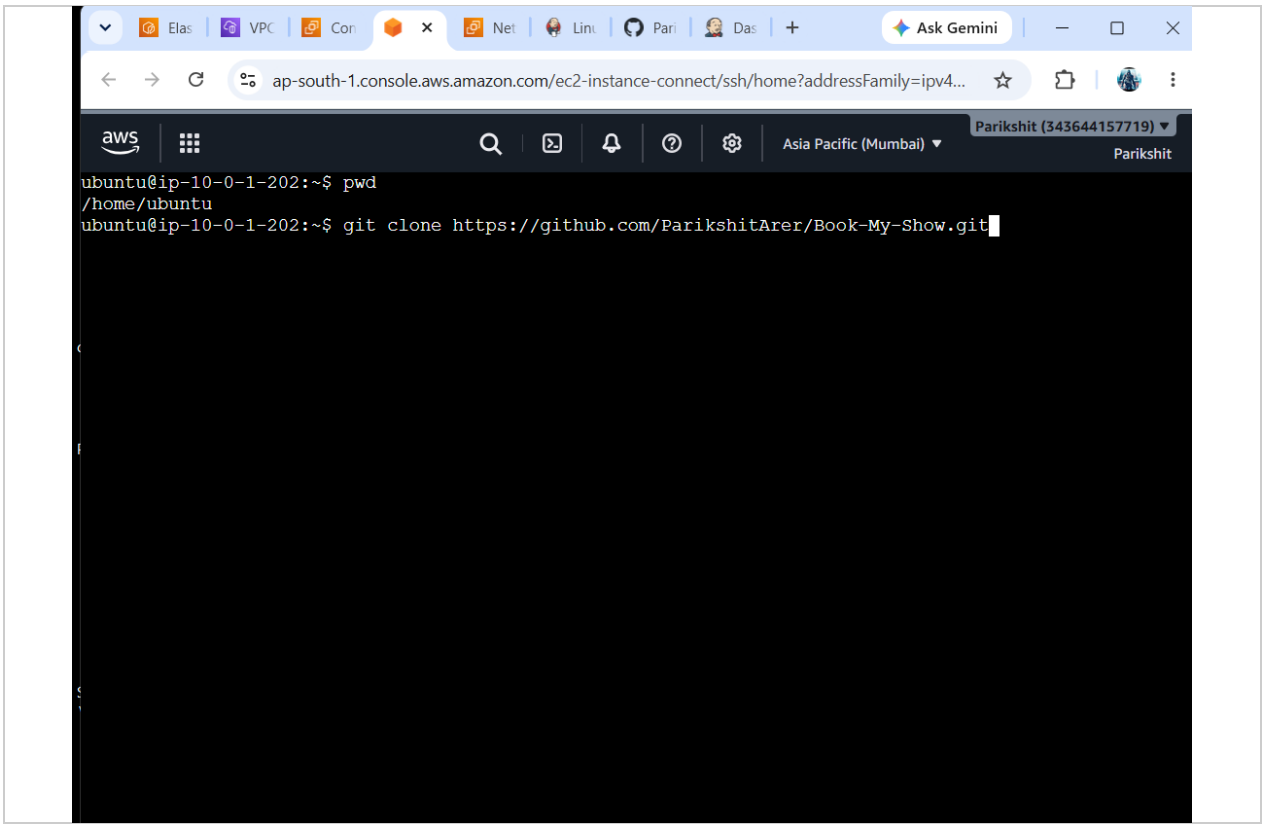



*Figure 6 — Cloning the Book-My-Show repository onto the Jenkins EC2 instance.*

## 5.1 Multi-Stage Dockerfile

The application is packaged with a multi-stage Dockerfile. The build stage compiles the app with Node 18 and installs dependencies; the production stage copies only the compiled build output into a lightweight nginx:alpine image, keeping the final image small and free of build tooling.

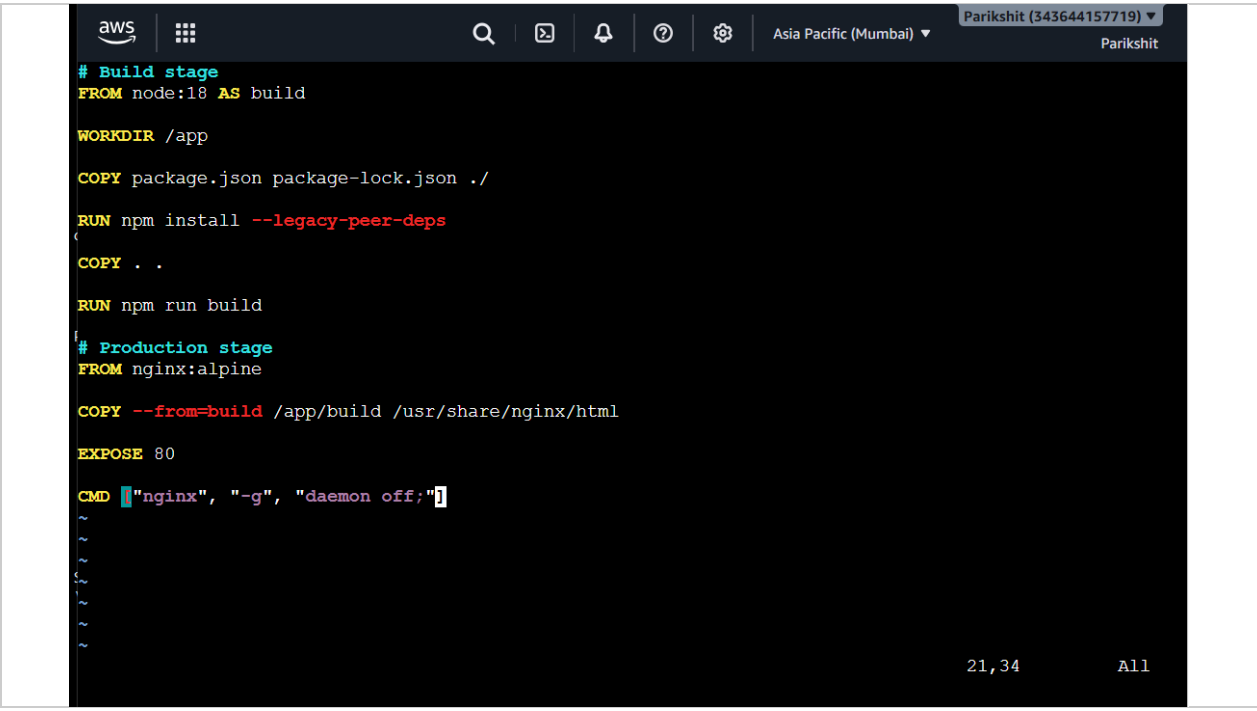



*Figure 7 — Multi-stage Dockerfile: Node 18 build stage, nginx:alpine production stage.*

## 5.2 Toolchain Versions

Docker, Git, AWS CLI, and kubectl versions were confirmed on the build host before wiring up the pipeline.

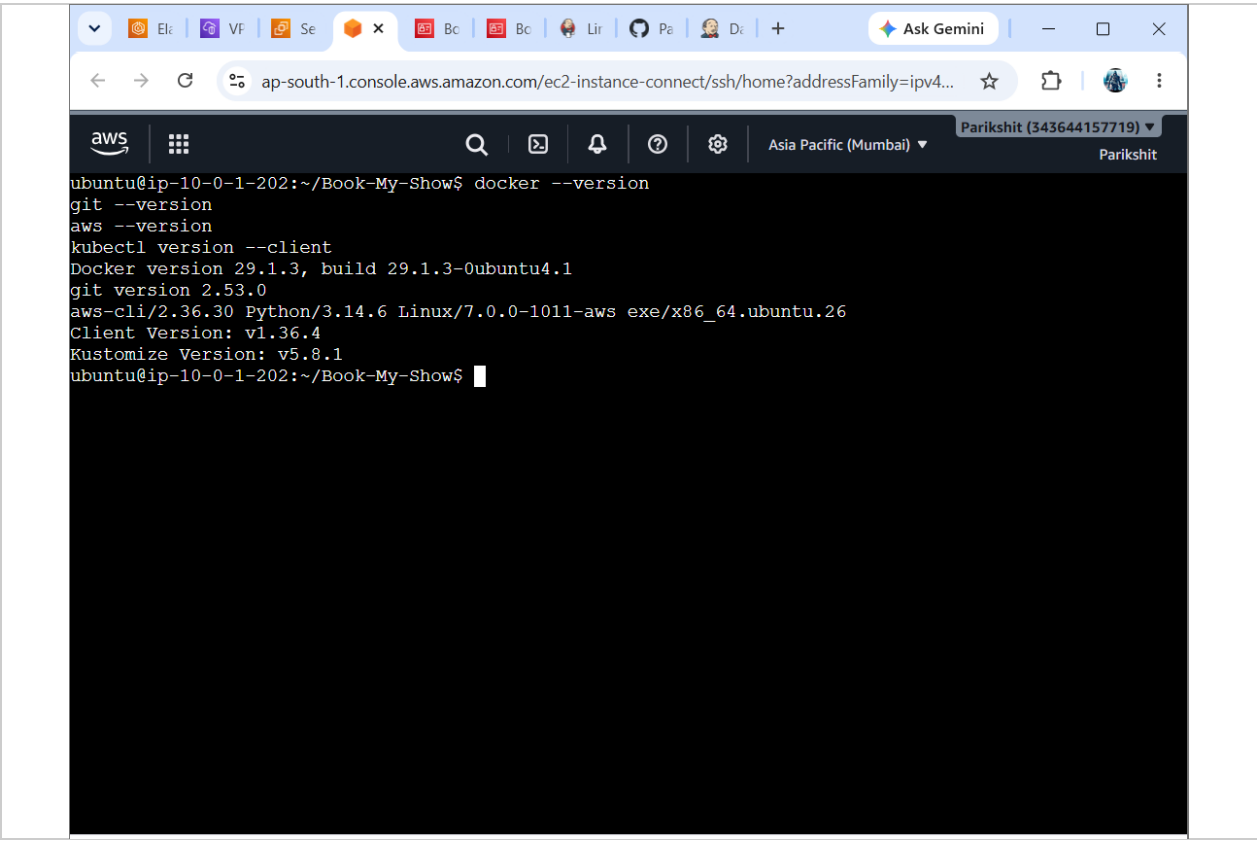



*Figure 8 — Confirmed toolchain: Docker 29.1.3, Git 2.53.0, AWS CLI 2.36.30, kubectl v1.36.4.*

# 6. CI/CD Pipeline — Jenkins

The BookMyShow-CI-CD Jenkins pipeline runs eight stages end to end: Checkout, Verify Files, Docker Build, ECR Login, Push to ECR, Ansible Deploy to EKS, Verify EKS Deployment, and Deployment Validation, wrapped by a declarative post-actions stage. A full run completes in roughly 38 seconds.

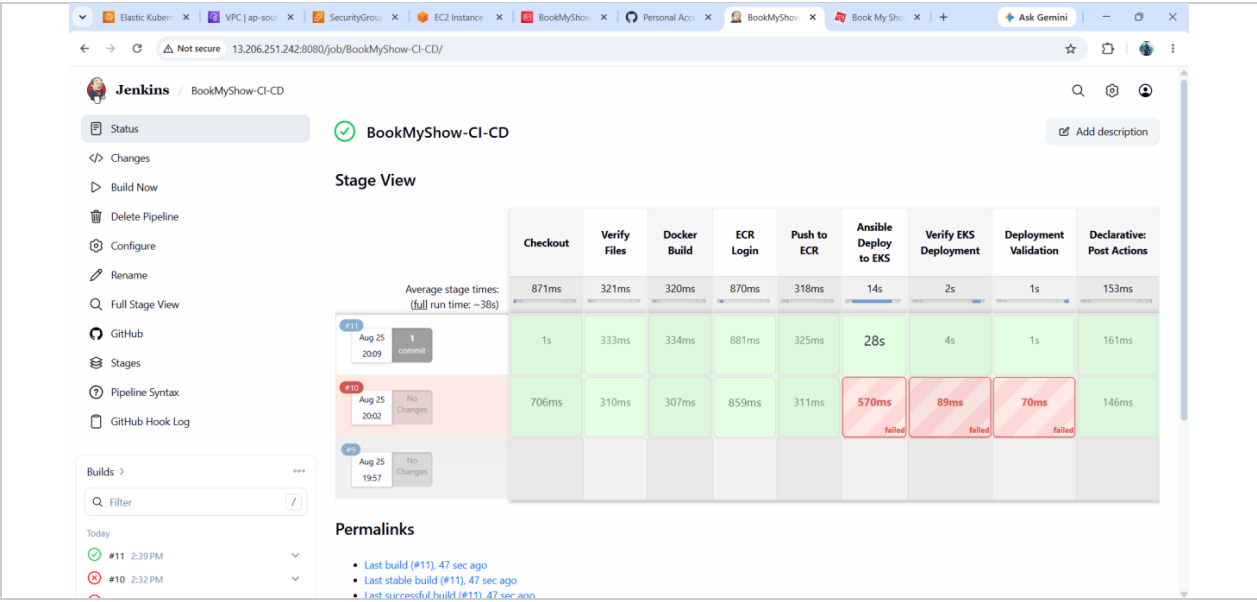


*Figure 9 — Jenkins Stage View for BookMyShow-CI-CD — 8-stage pipeline, build #11 succeeded after fixing build #10.*

Build #11 was triggered automatically by a commit titled “Add Ansible deployment automation” pushed to the main branch, and completed successfully in 38 seconds.

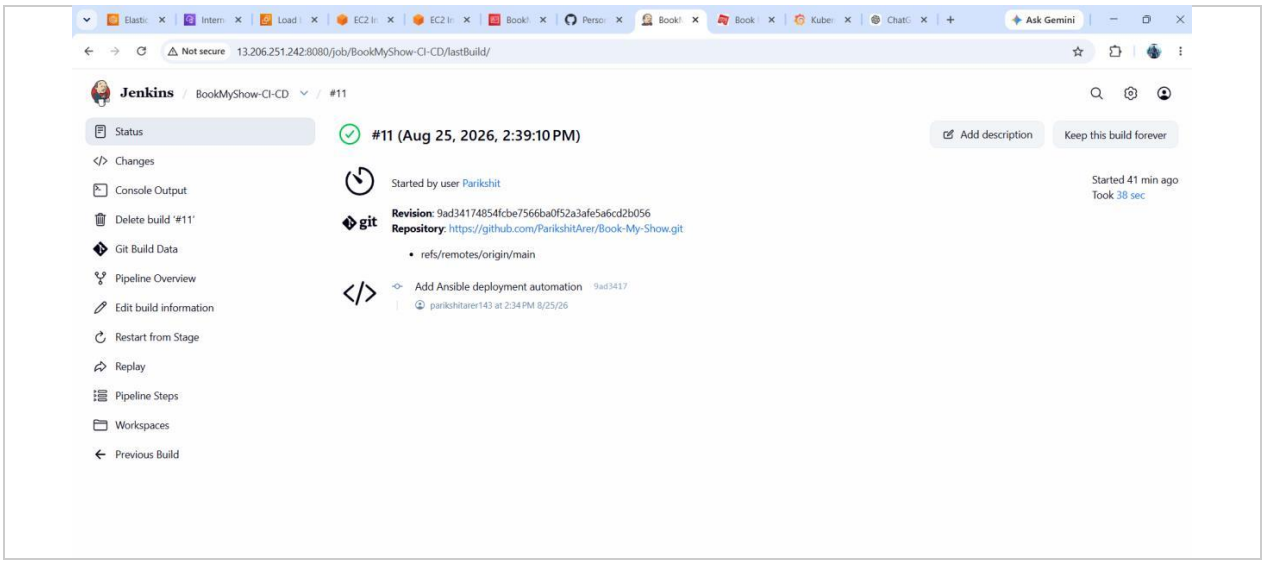



*Figure 10 — Build #11 detail — triggered by a GitHub commit, completed successfully.*

The pipeline is defined as Jenkins declarative script, checking out the main branch of the GitHub repository as its first stage.

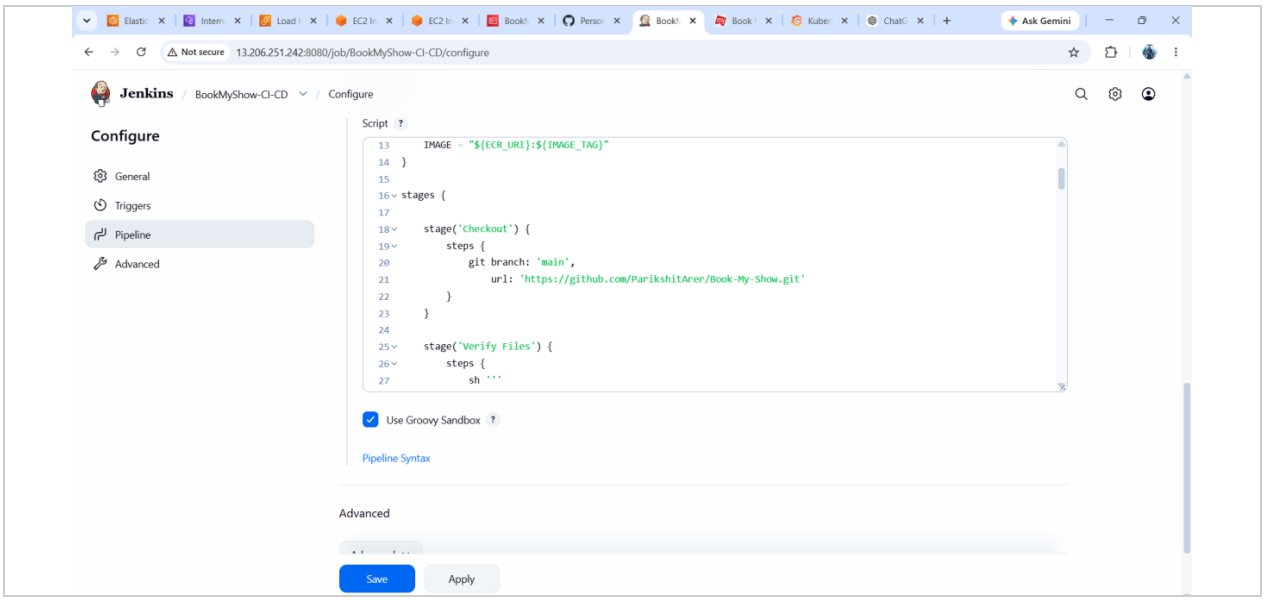



*Figure 11 — Jenkins pipeline script — Checkout and Verify Files stages, referencing the ECR image URI and tag.*

# 7. Container Registry — Amazon ECR

Each successful pipeline run builds a new Docker image and pushes it to the Jenkins build host and then to a private ECR repository, tagged with the Jenkins build number so every release is traceable back to its pipeline run.

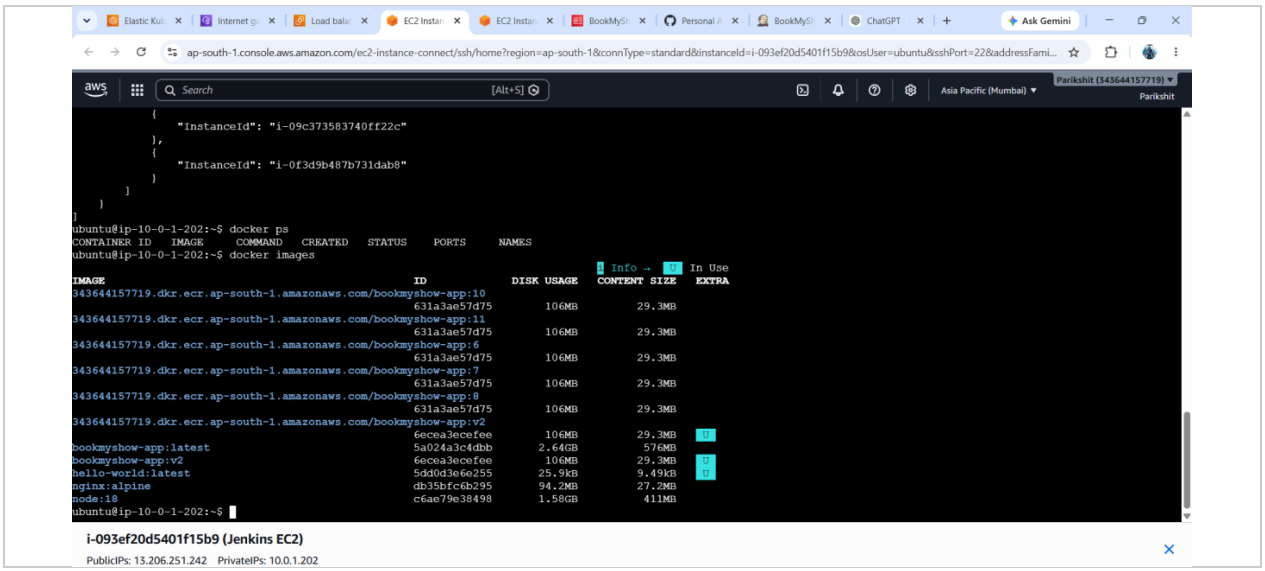



*Figure 12 — Docker images tagged with ECR URIs and Jenkins build numbers (6, 7, 8, 10, 11) on the build host.*

| | |
|---|---|
| **Repository Name** | bookmyshow-app |
| **Repository URI** | 343644157719.dkr.ecr.ap-south-1.amazonaws.com/bookmyshow-app |
| **Encryption** | AES-256 |
| **Tag Immutability** | Mutable |

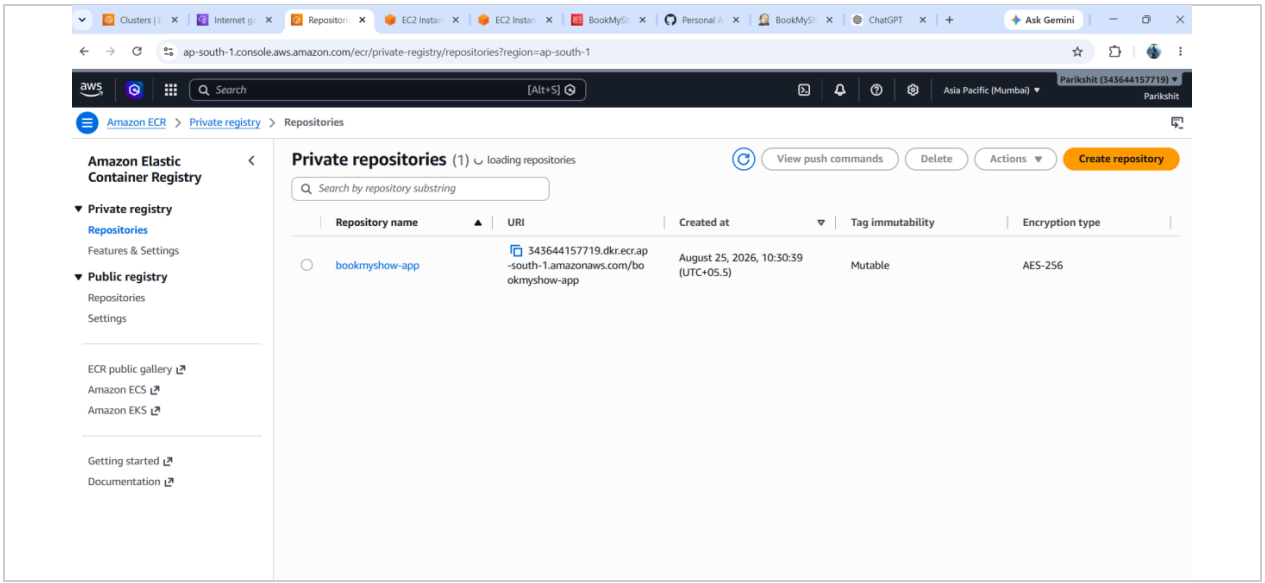



*Figure 13 — Amazon ECR private repository: bookmyshow-app, AES-256 encrypted.*

The repository holds the images pushed by each pipeline run, including builds 6–11 and an earlier v2 tag.

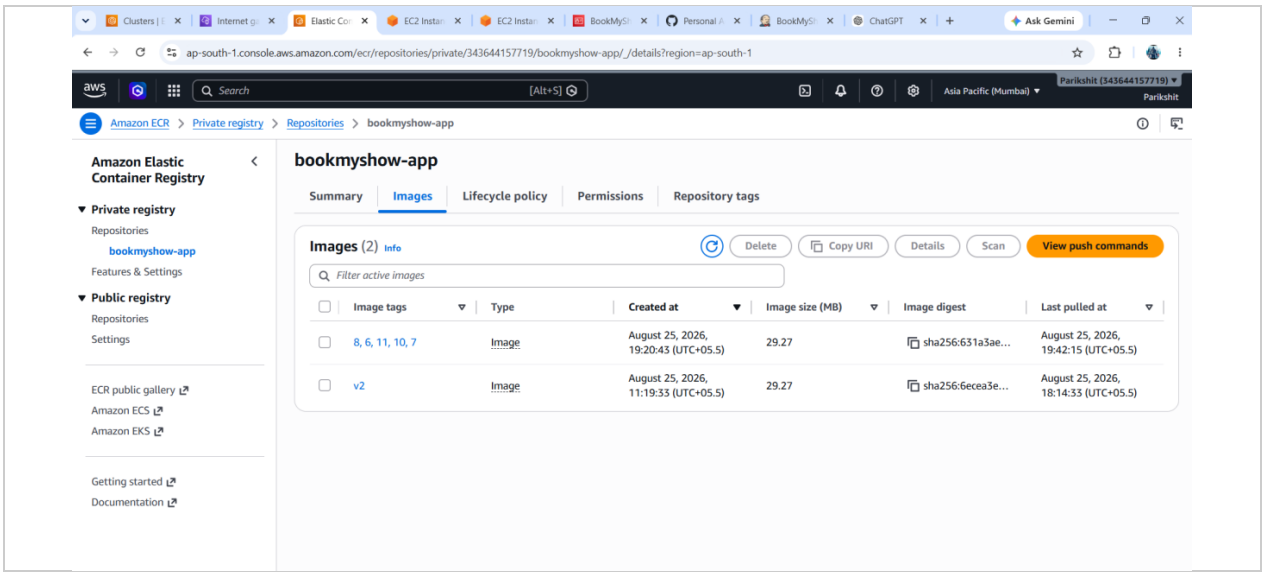



*Figure 14 — ECR image list — builds pushed by successive Jenkins runs.*

# 8. Container Orchestration — Amazon EKS

The BookMyShow-EKS cluster (Kubernetes v1.36) runs the application across a managed node group of Amazon Linux 2023 t3.small nodes, with autoscaling configured between 1 and 2 nodes.

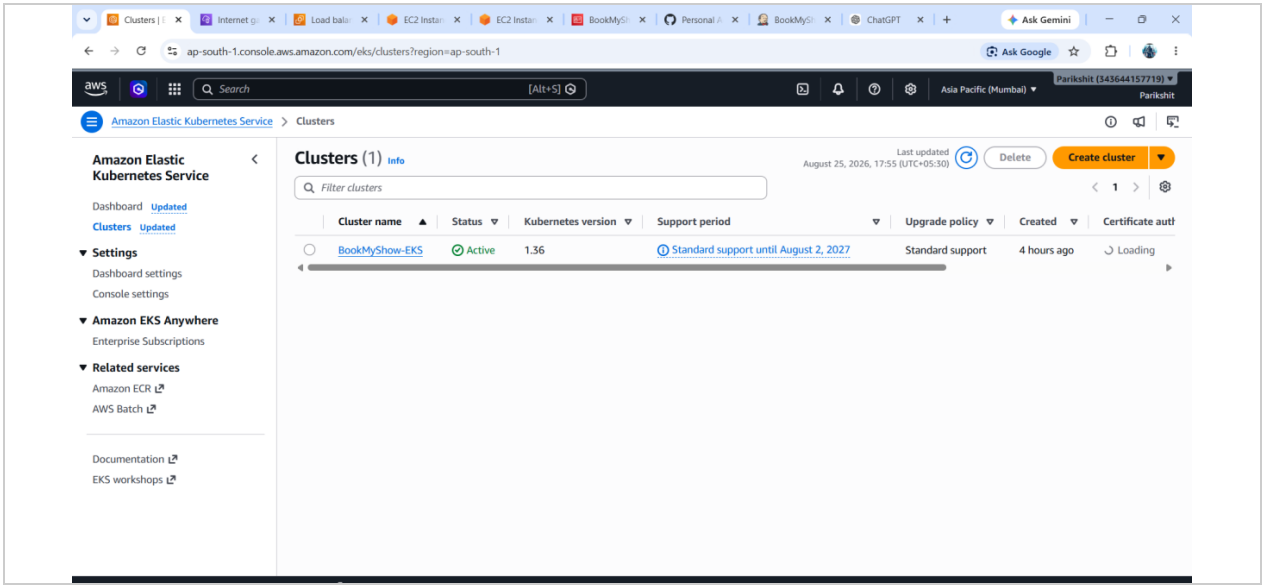



*Figure 15 — Amazon EKS cluster BookMyShow-EKS — Active, Kubernetes 1.36, standard support until Aug 2027.*

| | |
|---|---|
| **Node Group** | BookMyShow-EKS/node |
| **Kubernetes Version** | 1.36 |
| **AMI Type** | Amazon Linux 2023 (x86_64) Standard |
| **Instance Type** | t3.small |

| | |
|---|---|
| **Capacity Type** | On-Demand |
| **Desired / Min / Max Size** | 1 / 1 / 2 nodes |
| **Node IAM Role** | BookMyShow-EKS-Node-Role |

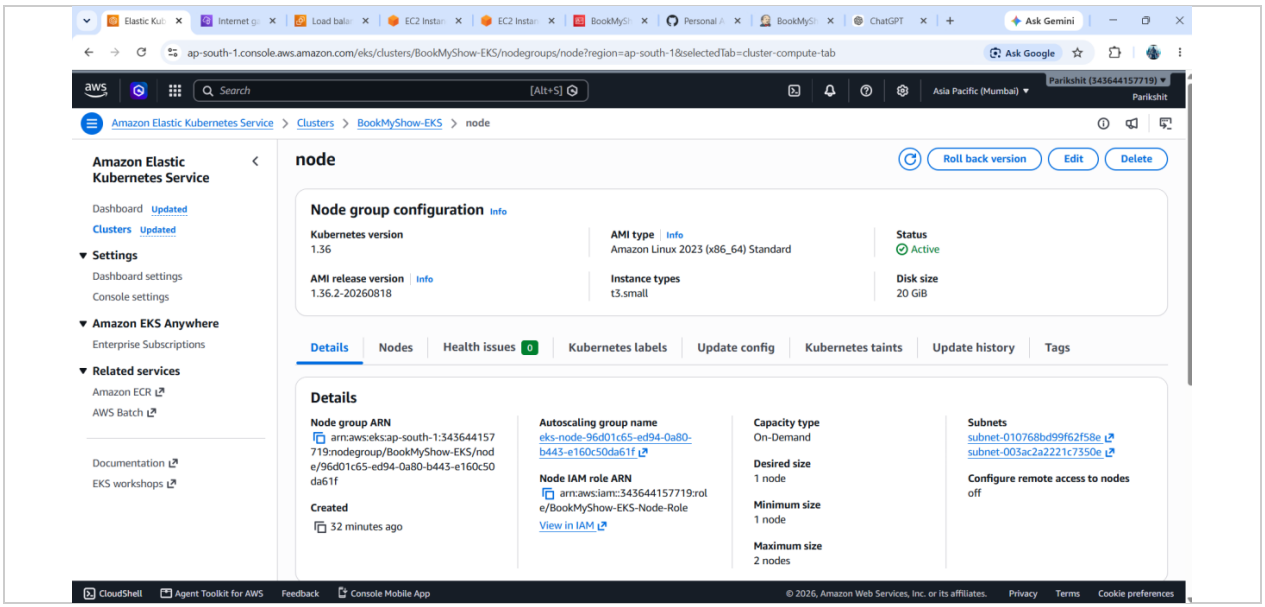



*Figure 16 — EKS managed node group configuration — Kubernetes 1.36, t3.small, autoscaling 1–2 nodes.*

kubectl confirms both worker nodes are Ready and the bms-app Deployment is running 2/2 replicas, one pod scheduled per node for resilience.

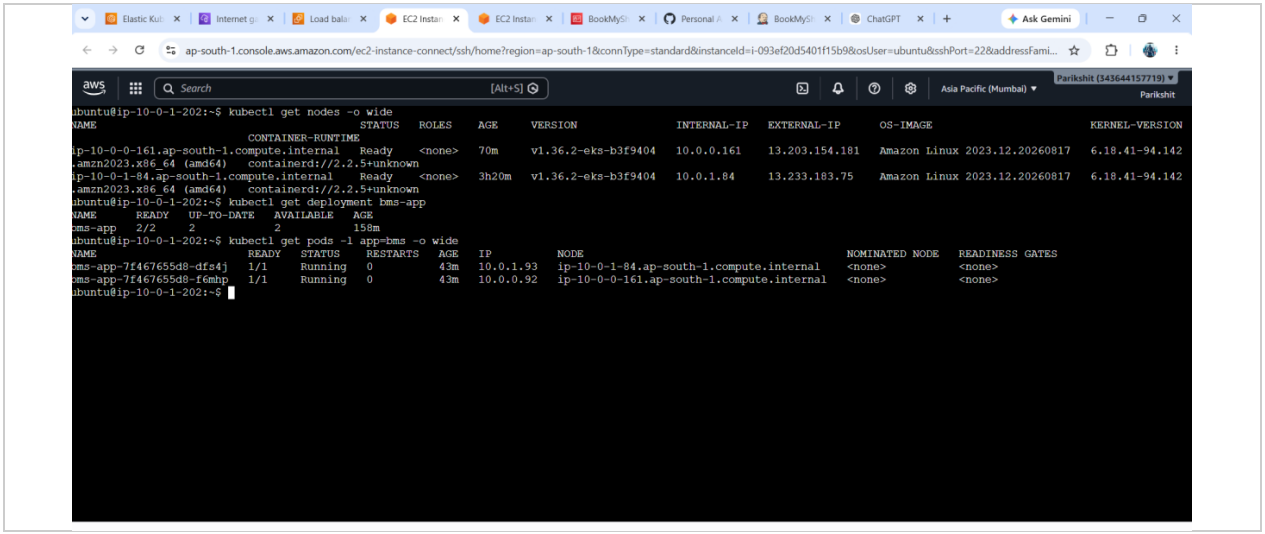



*Figure 17 — kubectl get nodes / get pods — 2 nodes Ready, 2/2 bms-app pods Running.*

# 9. Configuration Management — Ansible Deployment

Rather than running kubectl commands by hand, the pipeline's Ansible Deploy to EKS stage executes a playbook (ansible/deploy.yml) that updates the Deployment image, waits for the rollout to complete, and then reports back the deployment, pod, and service status — the same commands an operator would run, but scripted and repeatable.

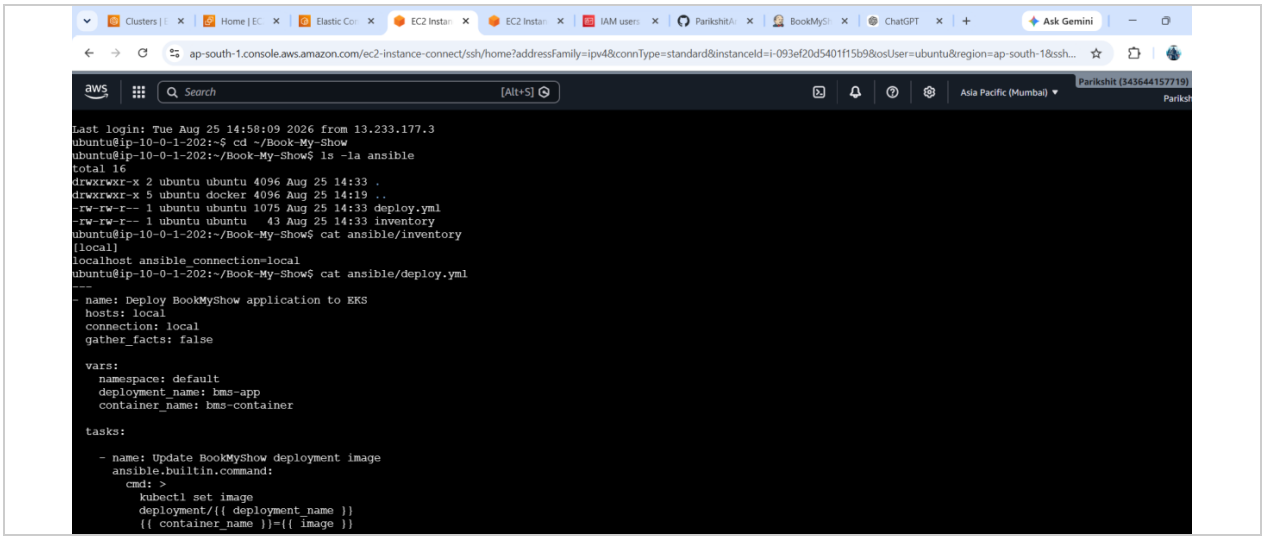



*Figure 18 — Ansible playbook (deploy.yml) and inventory on the Jenkins host: local connection, bms-app deployment target.*

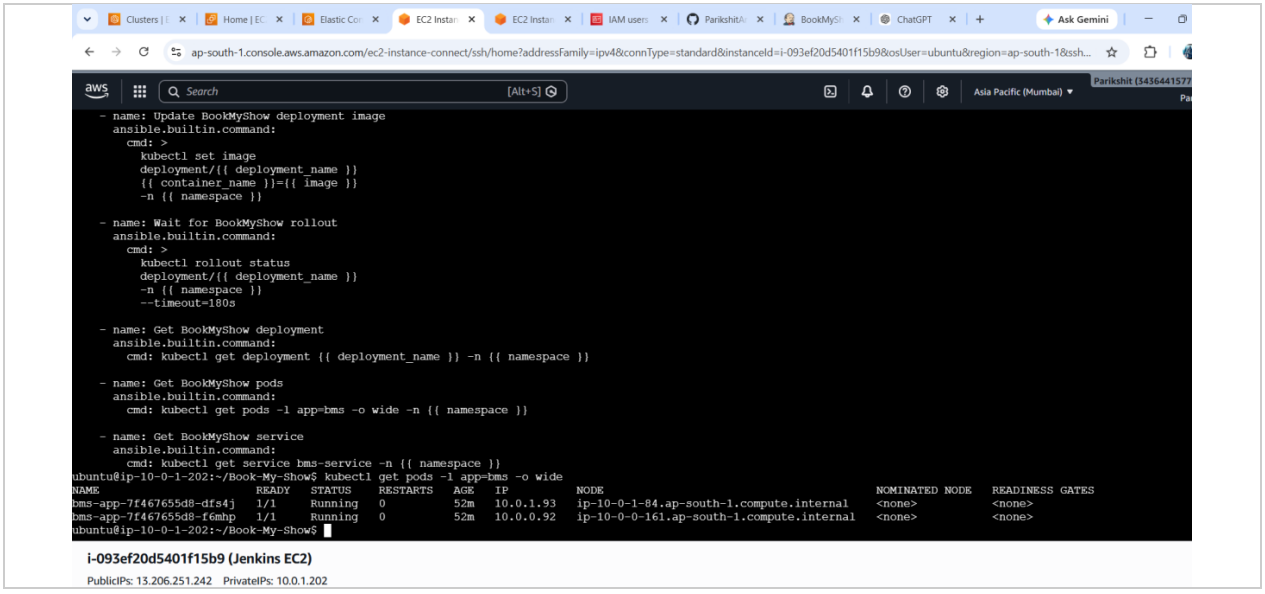



*Figure 19 — Full playbook task list — set image, wait for rollout, then fetch deployment/pod/service status.*

Executed from the Jenkins pipeline, the playbook reports the deployment as healthy, both pods Running, and the LoadBalancer service ready.

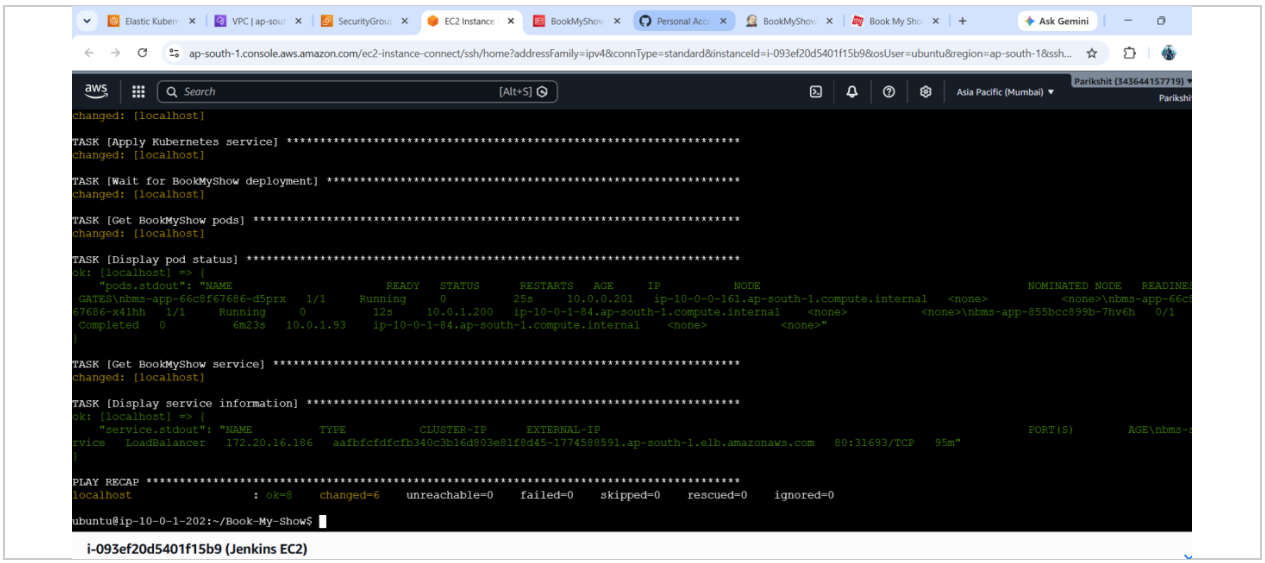



*Figure 20 — Ansible run output — Kubernetes service applied, deployment ready, pods Running, service exposed via LoadBalancer.*

# 10. Load Balancing & End-to-End Verification

The bms-service Kubernetes Service is of type LoadBalancer, provisioning a Classic Load Balancer that fronts the application pods across both Availability Zones. A curl request against the load balancer's DNS name returns HTTP 200, confirming the application tier is reachable end to end.

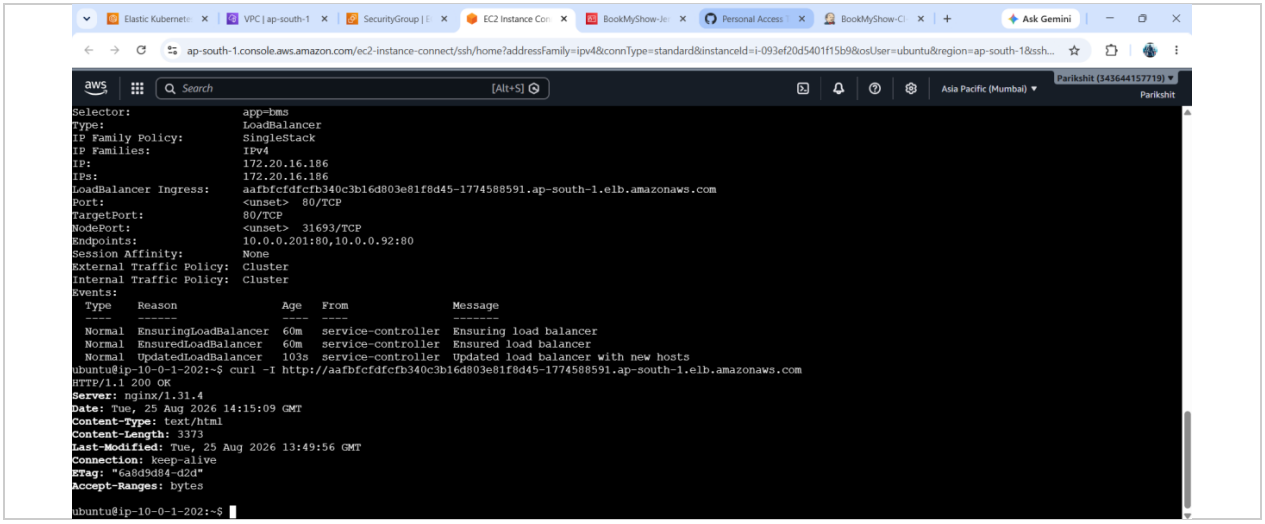



*Figure 21 — kubectl describe service and curl against the LoadBalancer endpoint — HTTP/1.1 200 OK.*

| | |
|---|---|
| **Load Balancer Type** | Classic |
| **Scheme** | Internet-facing |
| **Status** | 2 of 2 instances in service |
| **Availability Zones** | ap-south-1a, ap-south-1b |
| **VPC** | vpc-0fe3a4ca7e5fbd5e2 (Bookmyshow) |

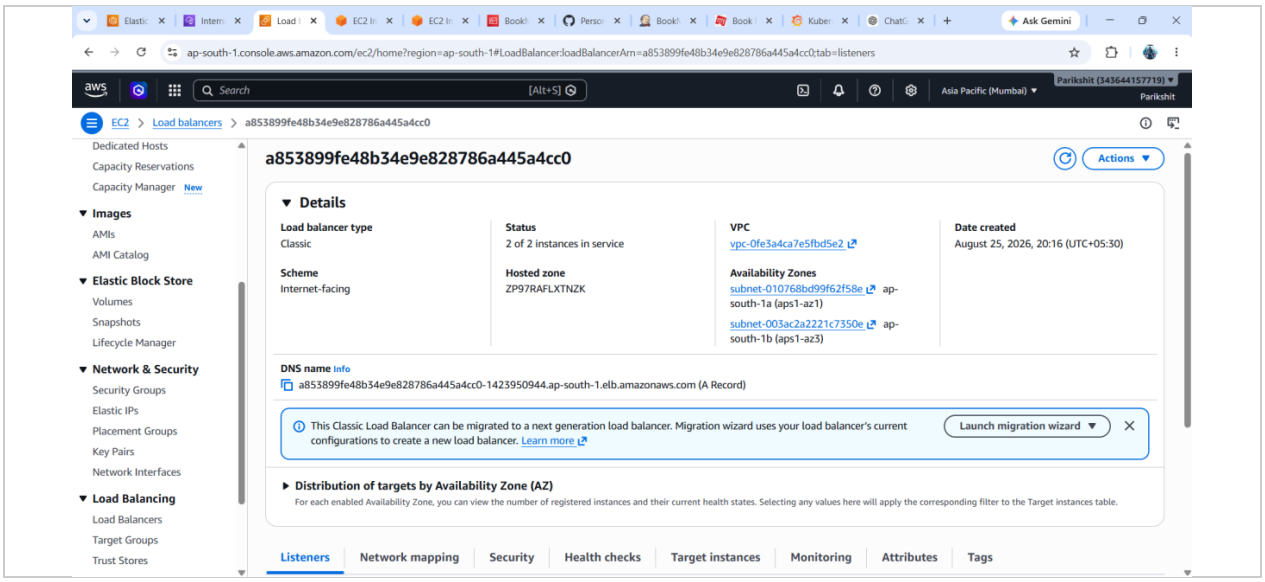



*Figure 22 — Classic Load Balancer provisioned by the Kubernetes Service — 2 of 2 instances healthy across both AZs.*

# 11. Identity & Access Management (IAM)

Access is split across four dedicated, purpose-scoped IAM roles rather than one broad role, following least-privilege practice: a role for the EKS cluster control plane, a role for EKS worker nodes, and two roles for the Jenkins EC2 host (build/deploy automation).

| | |
|---|---|
| **Role Name** | BookMyShow-Jenkins-Roles |
| **Purpose** | IAM role for the BookMyShow Jenkins EC2 server — build, ECR push, EKS deploy. |
| **Attached Policies** | 8 permission policies (incl. AmazonEC2Container... managed policies) |
| **Max Session Duration** | 1 hour |

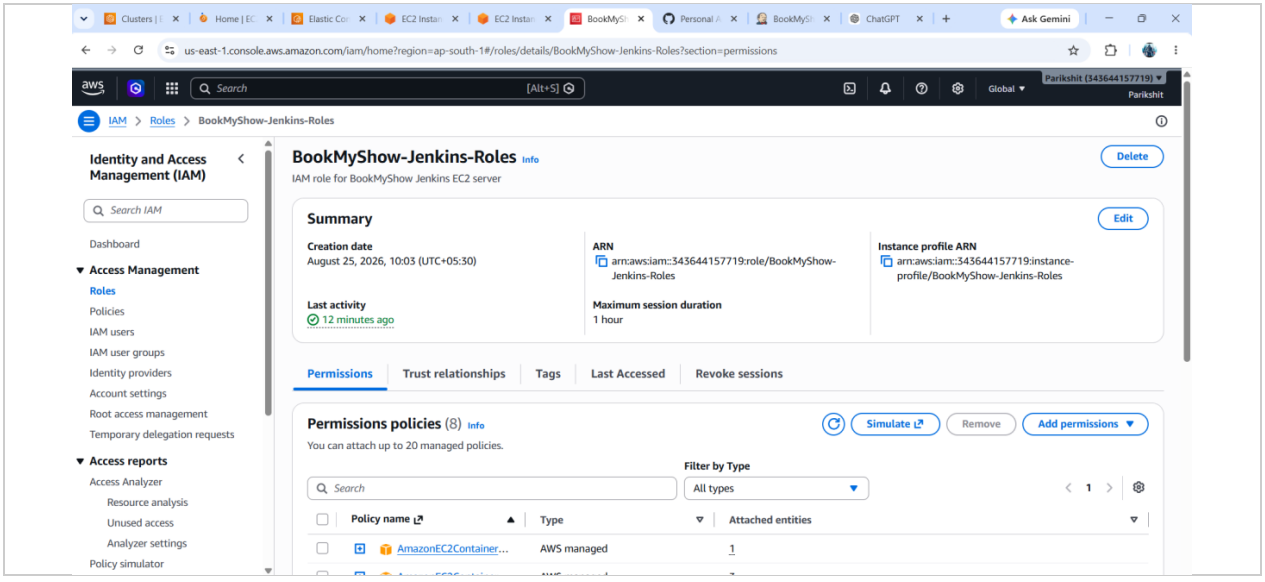


*Figure 23 — BookMyShow-Jenkins-Roles — IAM role assumed by the Jenkins EC2 server, 8 attached policies.*

Four BookMyShow-scoped roles cover the cluster, the node group, and the Jenkins host — each trusted only by the AWS service that needs it (eks or ec2).

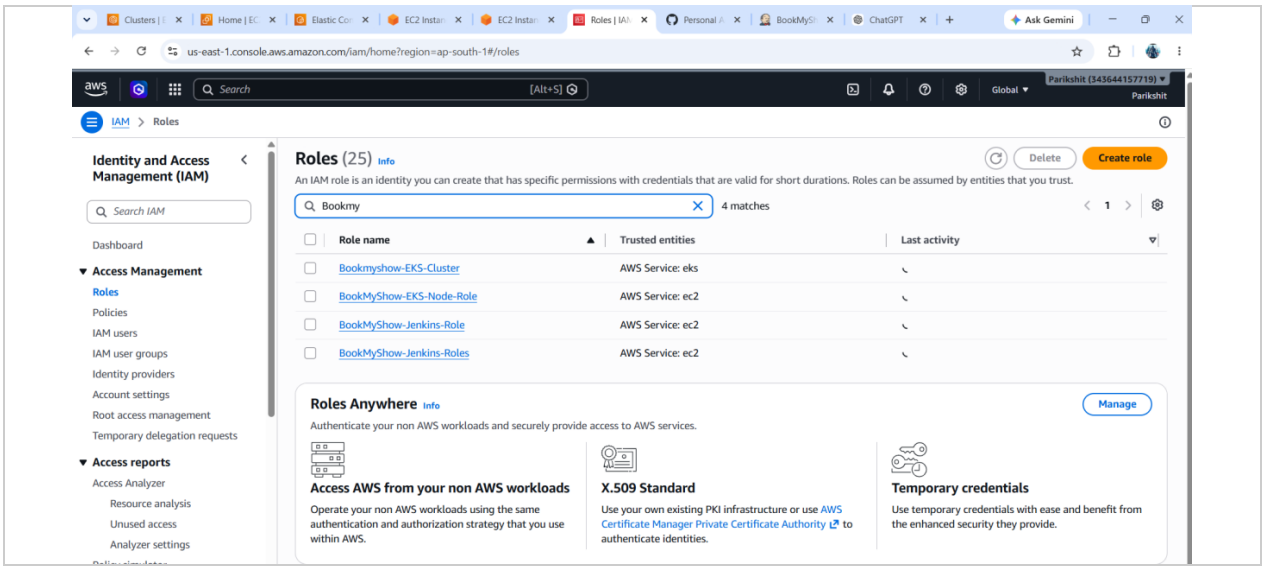



*Figure 24 — IAM roles scoped to BookMyShow: EKS-Cluster, EKS-Node-Role, Jenkins-Role, and Jenkins-Roles.*

# 12. Source Repository — GitHub

The pipeline's source of truth is the ParikshitArer/Book-My-Show repository (forked from staragile2016/Book-My-Show), holding the application code, the Jenkinsfile, the Kubernetes manifests (deployment.yml, service.yml), and the Ansible automation added for this deployment.

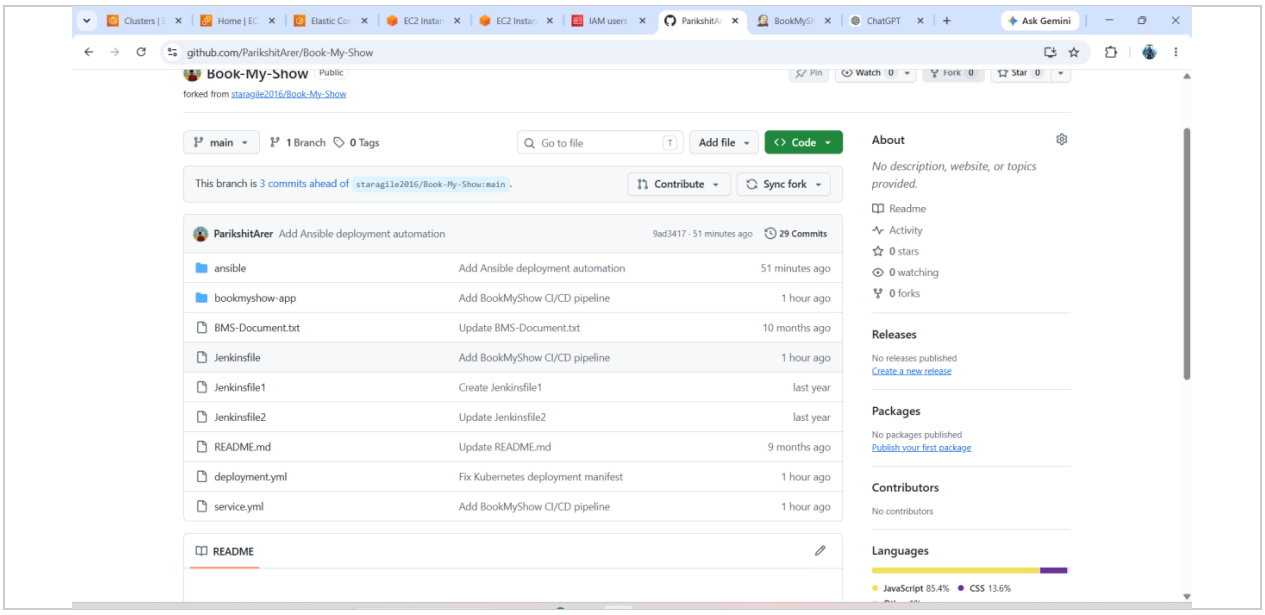



*Figure 25 — GitHub repository structure: ansible/, bookmyshow-app/, Jenkinsfile, deployment.yml, service.yml.*

# 13. Monitoring & Observability (Prometheus & Grafana)

Prometheus scrapes cluster and workload metrics, which are then visualized in Grafana using the standard kubernetes-mixin dashboard set — API server health, cluster-wide compute resources, and per-node/pod resource usage.

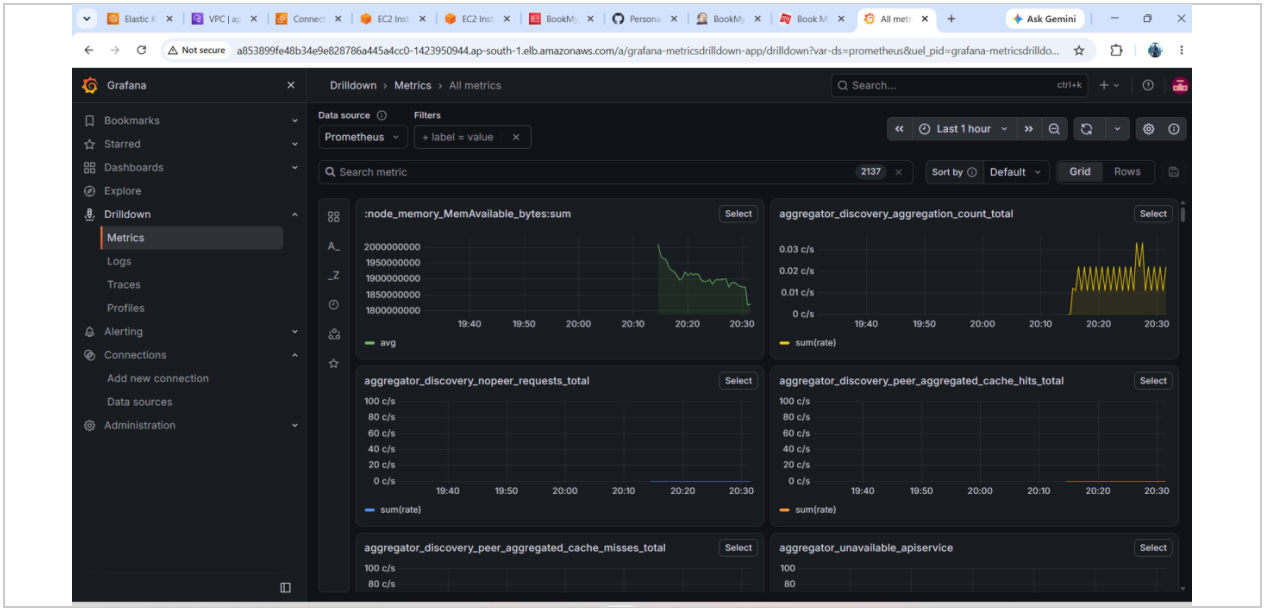



*Figure 26 — Grafana Metrics explorer, browsing raw Prometheus series such as aggregator and node-memory metrics.*

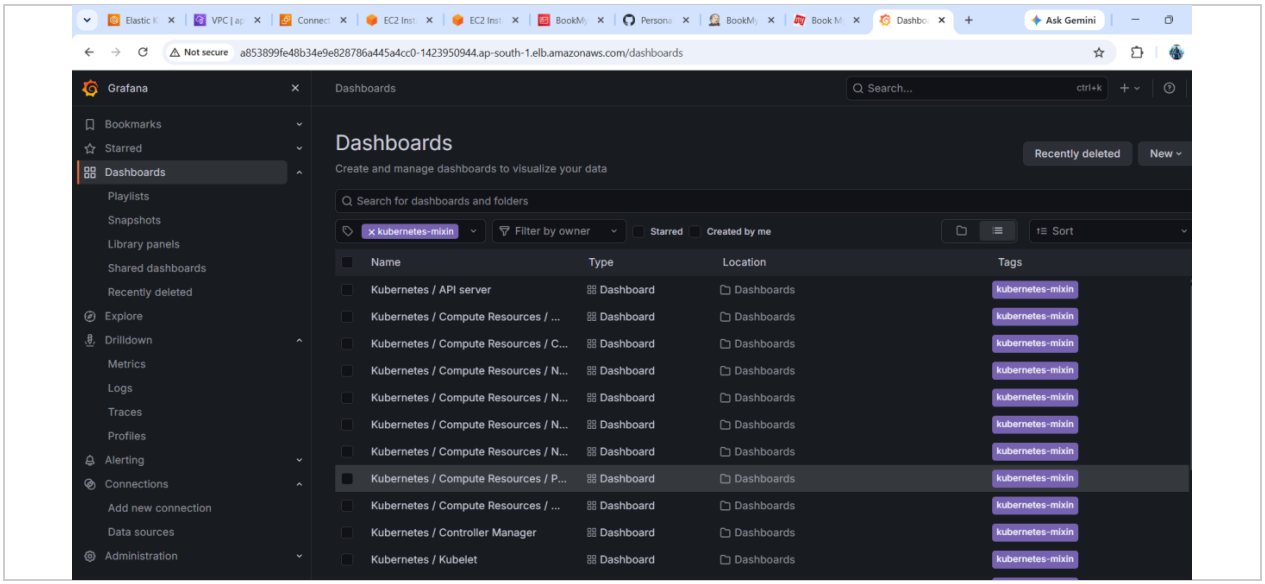



*Figure 27 — Grafana's kubernetes-mixin dashboard library: API server, Compute Resources, Controller Manager, Kubelet, and more.*

The Kubernetes / API server dashboard reports 100% availability over the observed window, with request, error, and duration SLIs all healthy.

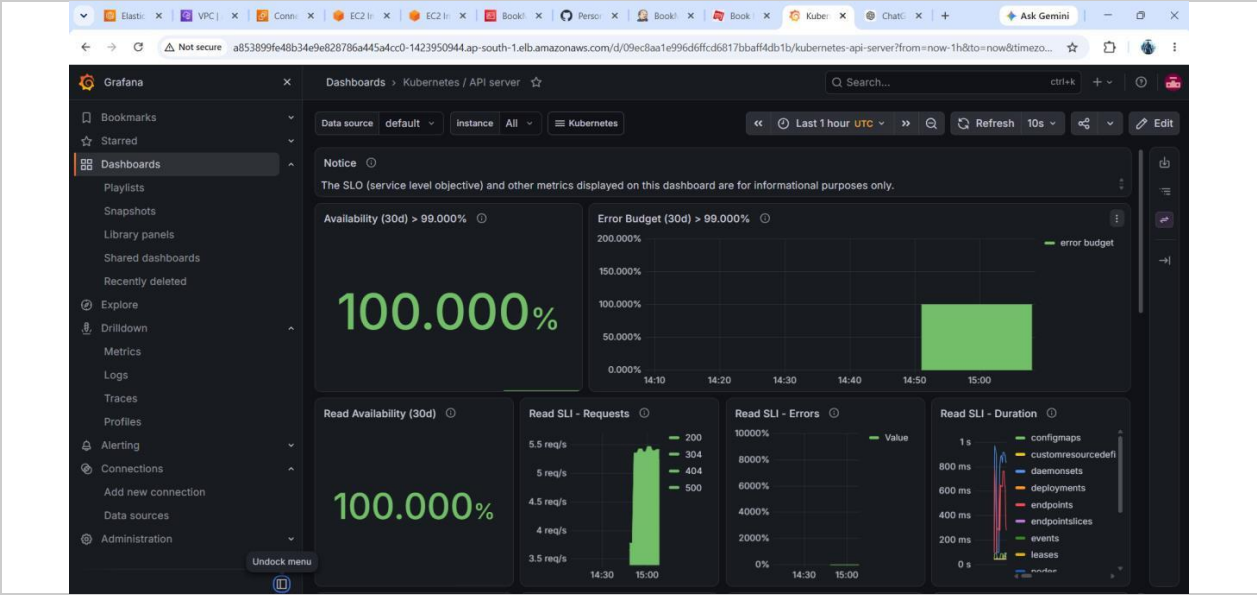



*Figure 28 — Kubernetes / API server dashboard — 100.000% availability, healthy read SLIs.*

At the cluster level, CPU utilization sits at 4.4% and memory utilization at 61.7% of allocatable capacity — comfortable headroom for the current workload.

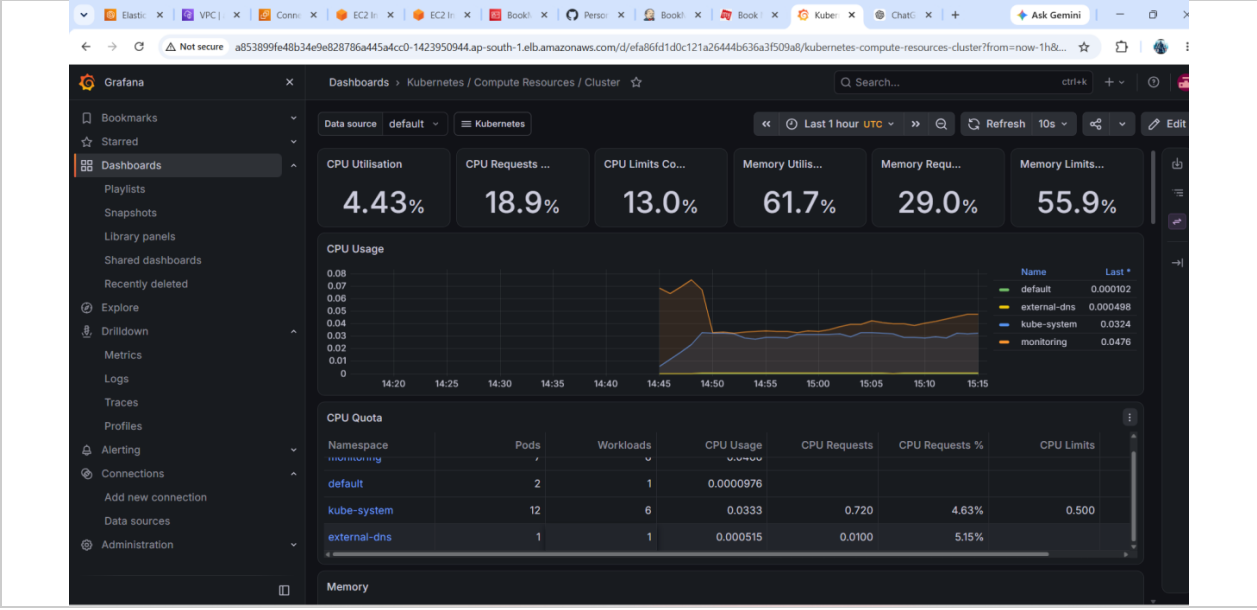



*Figure 29 — Kubernetes / Compute Resources / Cluster dashboard — CPU and memory utilization by namespace.*

Per-node memory usage (with and without cache) is tracked against max capacity and max allocatable, confirming the bms-app and monitoring pods stay well within their memory quotas.

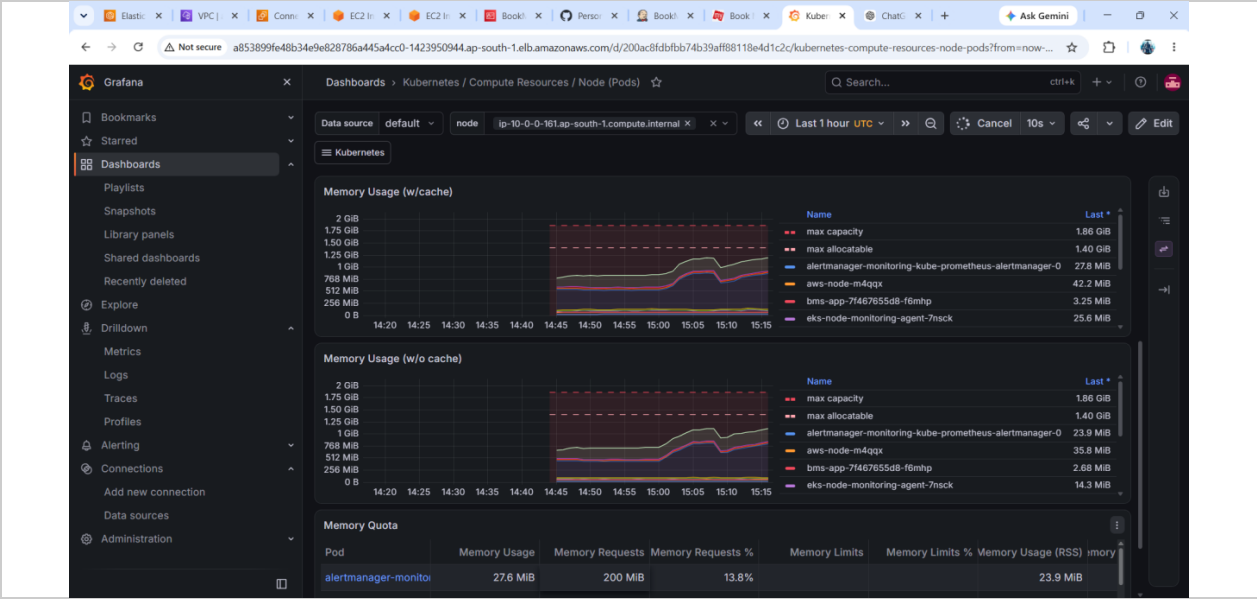



*Figure 30 — Kubernetes / Compute Resources / Node (Pods) dashboard — per-pod memory usage and quota.*

# 14. Summary & Outcomes

This deployment demonstrates a fully automated DevOps pipeline for a microservices-based ticketing application on AWS: a GitHub push triggers Jenkins, which builds a Docker image, pushes it to ECR, and hands off to Ansible for a zero-touch rollout to EKS — all backed by scoped IAM roles, VPC isolation, and live Prometheus/Grafana observability.

- Automated CI/CD Pipeline — an 8-stage Jenkins pipeline (~38s runtime) requires no manual intervention from commit to running pod.
- Scalable Microservices — Amazon EKS runs the app across an autoscaling node group (1–2 nodes), with 2/2 pods healthy and load-balanced.
- Configuration as Code — Ansible drives the Kubernetes rollout, replacing ad-hoc kubectl commands with a repeatable playbook.
- Least-Privilege Access — four dedicated IAM roles scope exactly the permissions each AWS service needs.
- Real-Time Monitoring — Prometheus and Grafana report 100% API-server availability and healthy resource headroom.
- Verified End-to-End — a direct curl against the load balancer confirms HTTP 200 from a freshly deployed release.

*Together, Jenkins, Docker, ECR, EKS, and Ansible — bound by VPC network isolation, scoped IAM roles, and Prometheus/Grafana observability — deliver on every business goal set out at the start of the project: frequent and reliable releases, reduced manual intervention, and real-time visibility into application health.*